RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 6b: Time Series Data Analysis and Forecast

Register Number: 241801177

Date: 09-10-2026

Dataset: Airline Passengers (monthly, 1949-1960)

Tools: Python, Pandas, statsmodels, Matplotlib, scikit-learn

Learning Objective

To understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement

An airline company keeps monthly records of the number of passengers travelling by air. As a Data Scientist, the task is to analyse the historical data, determine whether the series is stationary, apply suitable transformations, build an ARIMA forecasting model, predict passenger numbers for the test period and evaluate the accuracy of the forecasts.

Tasks 1-2: Download the dataset and read it with a DatetimeIndex

In [1]:

```
import os, urllib.request, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.seasonal import seasonal_decompose
from sklearn.metrics import mean_squared_error, mean_absolute_error
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')
pd.set_option('display.width', 110)

# ---- Task 1: download the airline-passengers dataset ----
FILE = 'airline-passengers.csv'
URL = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv'
if not os.path.exists(FILE):
    try: urllib.request.urlretrieve(URL, FILE)
    except Exception: print('Please place airline-passengers.csv in the working folder')
print("Task 1: dataset available:", os.path.exists(FILE))

# ---- Task 2: read the CSV so that the time index becomes a DatetimeIndex ----
df = pd.read_csv(FILE, parse_dates=['Month'], index_col='Month')
df = df.asfreq('MS')                                  # monthly (month-start) frequency
print("\nTask 2: index type:", type(df.index).__name__, "| frequency:", df.index.freqstr)
```

Out [1]:

```
Task 1: dataset available: True

Task 2: index type: DatetimeIndex | frequency: MS
```

> **Observation:** The dataset is read with parse_dates and index_col, so the Month column becomes a DatetimeIndex. Setting the frequency to month-start (MS) tells the time series functions that the data is regular and monthly.

Task 3: Explore the dataset

In [2]:

```
# ---- Task 3: explore the dataset ----
print("\nTask 3: Shape:", df.shape, "| Column:", list(df.columns))
print("\nFirst 5 records:"); print(df.head())
print("\nLast 5 records:"); print(df.tail())
print("\nDataset information:"); df.info()
print("\nSummary statistics:"); print(df.describe().round(2).to_string())
print("\nMissing values:", df.isnull().sum().sum(), "| Duplicate months:", df.index.duplicated().sum())
print("Period covered:", df.index.min().strftime('%Y-%m'), "to", df.index.max().strftime('%Y-%m'))
print("\nAverage passengers per year:"); print(df['Passengers'].groupby(df.index.year).mean().round(1).to_string())
series = df['Passengers']
```

Out [2]:

```
Task 3: Shape: (144, 1) | Column: ['Passengers']

First 5 records:
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121

Last 5 records:
            Passengers
Month
1960-08-01         606
1960-09-01         508
1960-10-01         461
1960-11-01         390
1960-12-01         432

Dataset information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Freq: MS
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB

Summary statistics:
       Passengers
count      144.00
mean       280.30
std        119.97
min        104.00
25%        180.00
50%        265.50
75%        360.50
max        622.00

Missing values: 0 | Duplicate months: 0
Period covered: 1949-01 to 1960-12

Average passengers per year:
Month
1949    126.7
1950    139.7
1951    170.2
1952    197.0
1953    225.0
1954    238.9
1955    284.0
1956    328.2
1957    368.4
1958    381.0
1959    428.3
1960    476.2
```

> **Observation:** The dataset has 144 monthly records (January 1949 to December 1960) in one column, Passengers, in thousands. There are no missing values and no duplicate months. The mean is 280.3 passengers and the range is 104 to 622. The yearly average grows from 126.7 in 1949 to 476.2 in 1960, nearly four times higher.

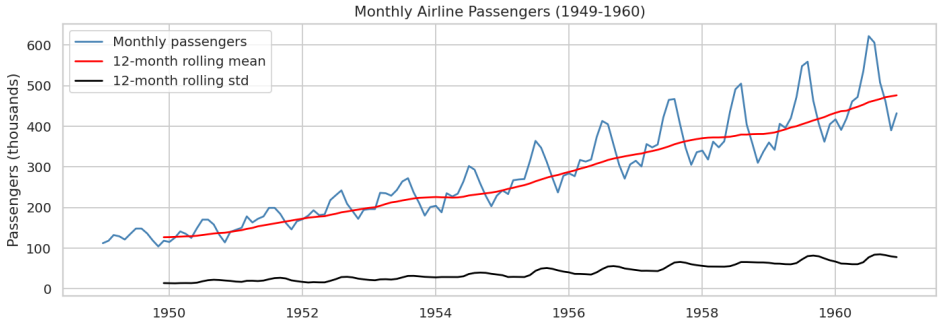
Task 4: Plot the series to inspect trend, seasonality and outliers

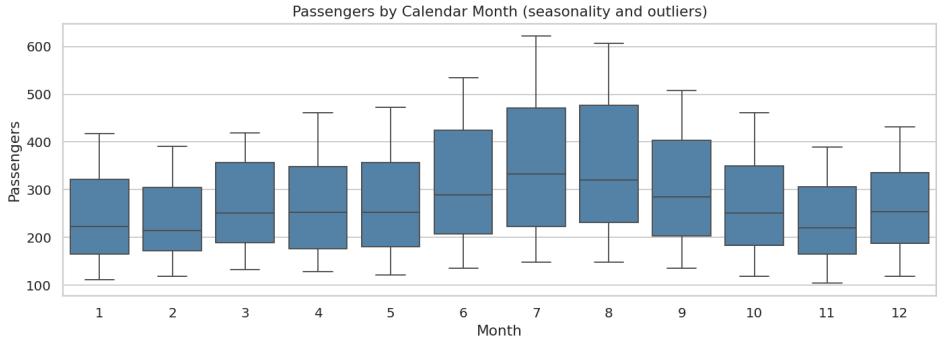
In [3]:

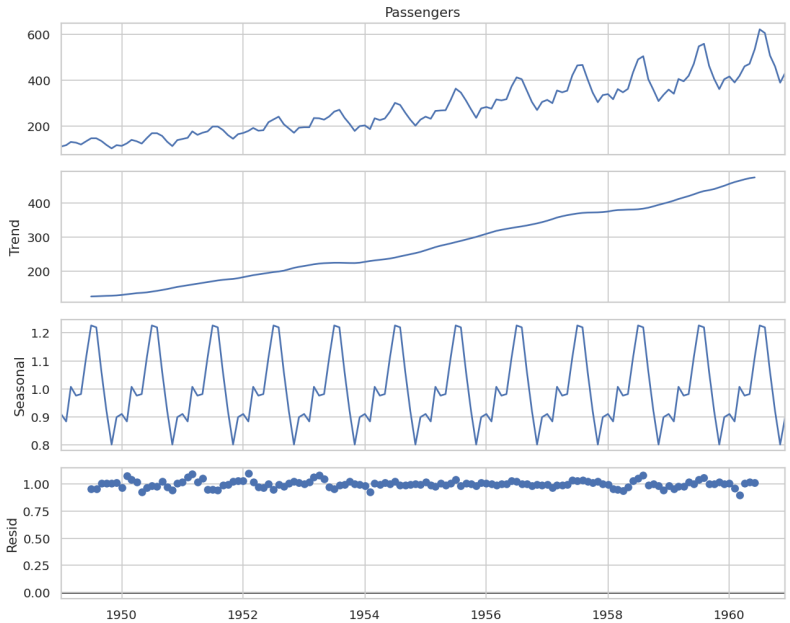
```
# ---- Task 4: plot the series (trend, seasonality, outliers) ----
fig, axes = plt.subplots(2, 1, figsize=(11, 8))
axes[0].plot(series, color='steelblue', label='Monthly passengers')
axes[0].plot(series.rolling(12).mean(), color='red', label='12-month rolling mean')
axes[0].plot(series.rolling(12).std(), color='black', label='12-month rolling std')
axes[0].set_title('Monthly Airline Passengers (1949-1960)'); axes[0].set_ylabel('Passengers (thousands)'); axes[0].legend()
sns.boxplot(x=series.index.month, y=series.values, color='steelblue', ax=axes[1])
axes[1].set_title('Passengers by Calendar Month (seasonality and outliers)'); axes[1].set_xlabel('Month'); axes[1].set_ylabel('Passengers')
plt.tight_layout(); plt.show()
dec = seasonal_decompose(series, model='multiplicative', period=12)
fig = dec.plot(); fig.set_size_inches(10, 8); plt.tight_layout(); plt.show()
print("\nTask 4: Peak month on average:", series.groupby(series.index.month).mean().idxmax(),
      "| Lowest month on average:", series.groupby(series.index.month).mean().idxmin())
```

Out [3]:

```
Task 4: Peak month on average: 7 | Lowest month on average: 11
```







Observation: The plot shows a clear upward trend and a repeating yearly seasonal pattern. July has the highest average and November the lowest. The seasonal swings get larger as the level rises (the rolling standard deviation increases), which means the seasonality is multiplicative, and the decomposition confirms this with seasonal factors of about 0.8 to 1.2. The boxplots show no isolated outliers, so no data points need to be removed.

Task 5: Stationarity check - Augmented Dickey-Fuller (ADF) test

In [4]:

```
# ---- ADF helper ----
def adf_test(s, name):
    stat, p, lags, nobs, crit, _ = adfuller(s.dropna(), autolag='AIC')
    print(f"ADF test on {name}")
    print(f"  ADF statistic    : {stat:.4f}")
    print(f"  p-value          : {p:.6f}")
    print(f"  Lags used        : {lags}")
    print(f"  Observations     : {nobs}")
    for k, v in crit.items(): print(f"  Critical value {k:>3s}: {v:.4f}")
    print("  Result           :", "STATIONARY (p < 0.05, reject H0)" if p < 0.05 else "NON-STATIONARY (p >= 0.05, cannot reject H0)")
    return p

# ---- Task 5: stationarity check on the original series ----
print("\nTask 5:"); p0 = adf_test(series, 'the original series')
```

Out [4]:

```
Task 5:
ADF test on the original series
  ADF statistic    : 0.8154
  p-value          : 0.991880
  Lags used        : 13
  Observations     : 130
  Critical value  1%: -3.4817
  Critical value  5%: -2.8840
  Critical value 10%: -2.5788
  Result           : NON-STATIONARY (p >= 0.05, cannot reject H0)
```

Observation: The ADF statistic (0.8154) is above all critical values, and the p-value is 0.9919, far above 0.05. The null hypothesis of a unit root cannot be rejected, so the original series is non-stationary, as expected from its trend and growing seasonality.

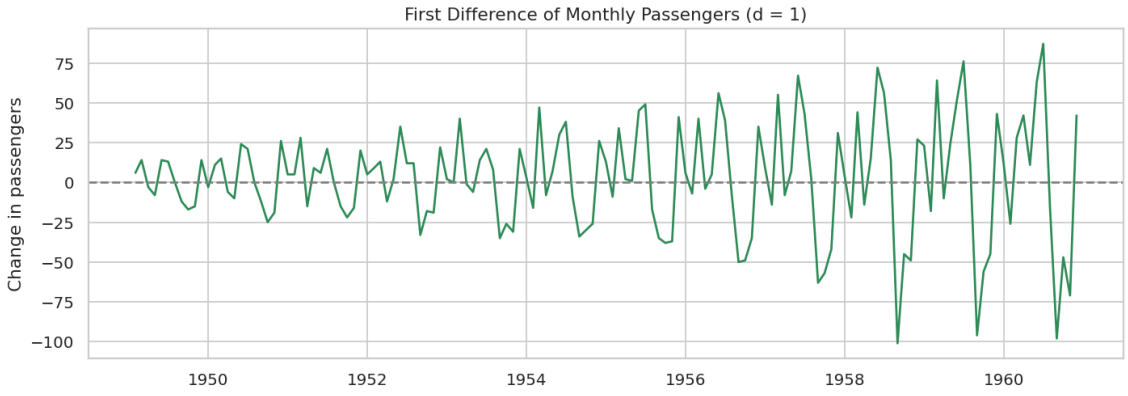
Task 6: Make the series stationary - first-order differencing

In [5]:

```
# ---- Task 6: first-order differencing ----
diff1 = series.diff().dropna()
print("\nTask 6: first difference (first 5 values):"); print(diff1.head().to_string())
plt.figure(figsize=(11, 4)); plt.plot(diff1, color='seagreen'); plt.axhline(0, color='grey', linestyle='--')
plt.title('First Difference of Monthly Passengers (d = 1)'); plt.ylabel('Change in passengers'); plt.tight_layout()
plt.show()
```

Out [5]:

```
Task 6: first difference (first 5 values):
Month
1949-02-01     6.0
1949-03-01    14.0
1949-04-01    -3.0
1949-05-01    -8.0
1949-06-01    14.0
Freq: MS
```



Observation: First-order differencing removes the trend: the differenced series moves around zero. The size of the fluctuations still increases with time, so the variance is not constant.

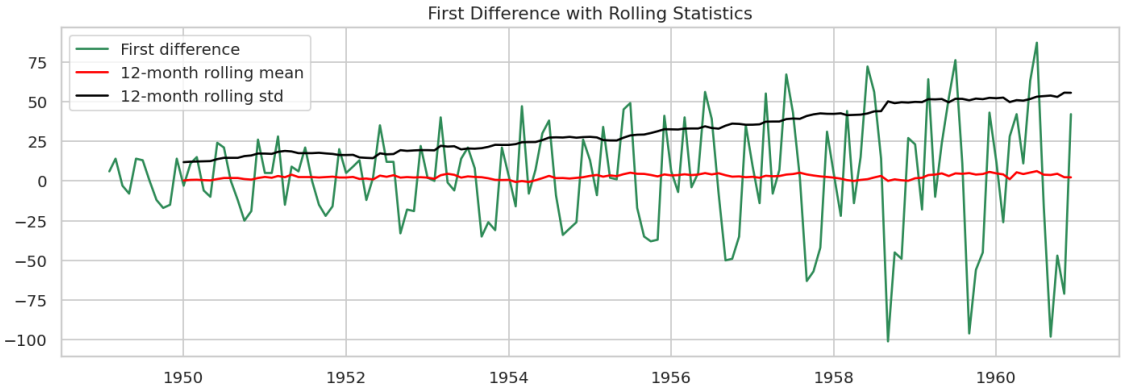
Task 7: Re-run ADF on the differenced series and visualise it

In [6]:

```
# ---- Task 7: re-run ADF on the differenced series and visualise it ----
print("\nTask 7:"); p1 = adf_test(diff1, 'the first difference (d = 1)')
plt.figure(figsize=(11, 4))
plt.plot(diff1, color='seagreen', label='First difference')
plt.plot(diff1.rolling(12).mean(), color='red', label='12-month rolling mean')
plt.plot(diff1.rolling(12).std(), color='black', label='12-month rolling std')
plt.title('First Difference with Rolling Statistics'); plt.legend(); plt.tight_layout(); plt.show()
```

Out [6]:

```
Task 7:
ADF test on the first difference (d = 1)
  ADF statistic    : -2.8293
  p-value          : 0.054213
  Lags used        : 12
  Observations     : 130
  Critical value  1%: -3.4817
  Critical value  5%: -2.8840
  Critical value 10%: -2.5788
  Result           : NON-STATIONARY (p >= 0.05, cannot reject H0)
```



Observation: The ADF statistic is -2.8293 and the p-value is 0.0542. This is just above 0.05: the series would be stationary at the 10% level but not at the usual 5% level (critical value -2.8840). The rolling standard deviation also keeps rising, so the first difference is treated as still non-stationary and Task 8 is needed.

Task 8: Second difference (d = 2) or log transformation

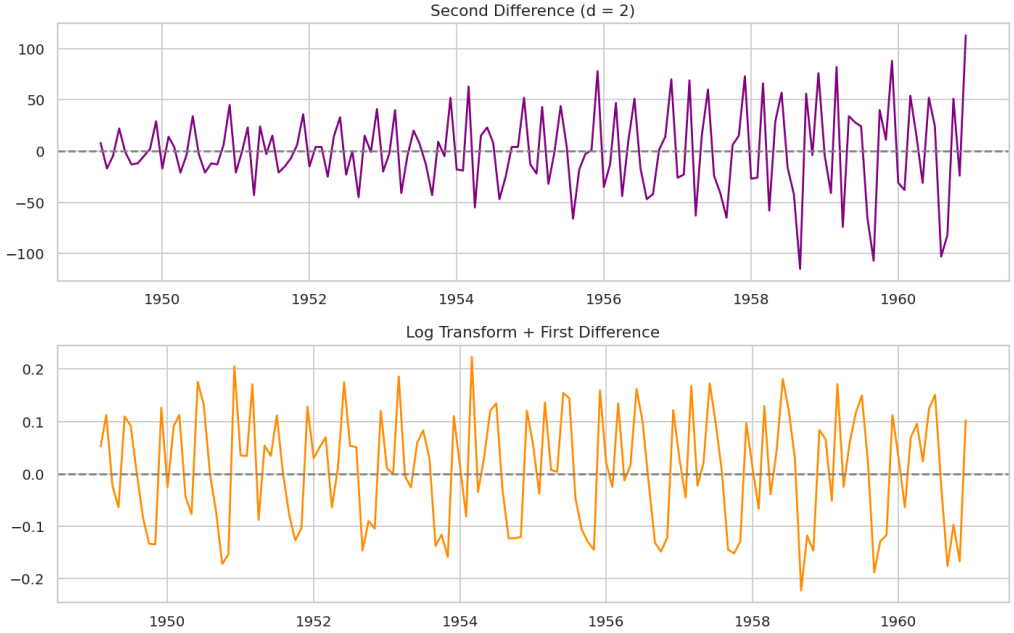
In [7]:

```
# ---- Task 8: if still non-stationary, try a second difference or a log transform ----
diff2 = series.diff().diff().dropna()
log_diff1 = np.log(series).diff().dropna()
if p1 >= 0.05:
    print("\nTask 8: first difference is still non-stationary, so other options are tested.")
    p2 = adf_test(diff2, 'the second difference (d = 2)')
    p_log = adf_test(log_diff1, 'the log-transformed series with one difference')
    fig, axes = plt.subplots(2, 1, figsize=(11, 7))
    axes[0].plot(diff2, color='purple'); axes[0].axhline(0, color='grey', linestyle='--'); axes[0].set_title('Second Difference (d = 2)')
    axes[1].plot(log_diff1, color='darkorange'); axes[1].axhline(0, color='grey', linestyle='--'); axes[1].set_title('Log Transform + First Difference')
    plt.tight_layout(); plt.show()
else:
    print("\nTask 8: the first difference is already stationary, so no further step is needed.")
    p2 = None
d = 0 if p0 < 0.05 else (1 if p1 < 0.05 else 2)
print(f"\nDifferencing order used for ARIMA: d = {d}")
```

Out [7]:

```
Task 8: first difference is still non-stationary, so other options are tested.
ADF test on the second difference (d = 2)
  ADF statistic    : -16.3842
  p-value          : 0.000000
  Lags used        : 11
  Observations     : 130
  Critical value  1%: -3.4817
  Critical value  5%: -2.8840
  Critical value 10%: -2.5788
  Result           : STATIONARY (p < 0.05, reject H0)
ADF test on the log-transformed series with one difference
  ADF statistic    : -2.7171
  p-value          : 0.071121
  Lags used        : 14
  Observations     : 128
  Critical value  1%: -3.4825
  Critical value  5%: -2.8844
  Critical value 10%: -2.5790
  Result           : NON-STATIONARY (p >= 0.05, cannot reject H0)

Differencing order used for ARIMA: d = 2
```



Observation: The second difference is clearly stationary (ADF statistic -16.38, p-value about 0, far below the 1% critical value of -3.48). The log transformation with one difference gives a p-value of 0.0711, so it is still non-stationary at the 5% level. Therefore two differences are applied and d = 2 is used for the ARIMA model.

Task 9: Split into training and testing sets (original series)

In [8]:

```
# ---- Task 9: split into training and testing sets (original series, in time order) ----
n_train = int(len(series) * 0.80)
train, test = series.iloc[:n_train], series.iloc[n_train:]
print(f"\nTask 9: Train = {len(train)} months ({train.index.min():%Y-%m} to {train.index.max():%Y-%m})")
print(f"        Test  = {len(test)} months ({test.index.min():%Y-%m} to {test.index.max():%Y-%m})")
```

Out [8]:

```
Task 9: Train = 115 months (1949-01 to 1958-07)
        Test  = 29 months (1958-08 to 1960-12)
```

Observation: The split is in time order, without shuffling: the first 115 months (January 1949 to July 1958) are used for training and the last 29 months (August 1958 to December 1960) for testing. This is about an 80:20 split.

Task 10: Fit the ARIMA model on the training set

In [9]:

```
# ---- Task 10: fit ARIMA(p, d, q) on the training set (p and q chosen by lowest AIC) ----
rows = []
for p in range(4):
    for q in range(4):
        try:
            m = ARIMA(train, order=(p, d, q)).fit()
            rows.append((p, d, q, m.aic, m.bic))
        except Exception:
            pass
grid = pd.DataFrame(rows, columns=['p', 'd', 'q', 'AIC', 'BIC']).sort_values('AIC').reset_index(drop=True)
print("\nTask 10: Five best ARIMA orders by AIC:"); print(grid.head(5).round(2).to_string(index=False))
order = (int(grid.loc[0, 'p']), int(grid.loc[0, 'd']), int(grid.loc[0, 'q']))
print("Selected order (p, d, q):", order)
model = ARIMA(train, order=order).fit()
print(model.summary())
```

Out [9]:

```
Task 10: Five best ARIMA orders by AIC:
 p  d  q     AIC     BIC
 1  2  2 1054.89 1065.80
 1  2  3 1056.06 1069.69
 2  2  2 1056.17 1069.81
 0  2  3 1057.37 1068.28
 3  2  2 1057.71 1074.07
Selected order (p, d, q): (1, 2, 2)
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(1, 2, 2)   Log Likelihood                -523.445
Date:                Fri, 09 Oct 2026   AIC                           1054.889
```

```
Time:                        05:37:08   BIC                          1065.799
Sample:                    01-01-1949   HQIC                         1059.316
                         - 07-01-1958
Covariance Type:                 opg
================================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------------
ar.L1         -0.5137      0.123     -4.192      0.000      -0.754      -0.274
ma.L1         -0.0843     13.690     -0.006      0.995     -26.916      26.748
ma.L2         -0.9155     12.512     -0.073      0.942     -25.438      23.607
sigma2       590.7316   8099.367      0.073      0.942   -1.53e+04    1.65e+04
===================================================================================
Ljung-Box (L1) (Q):                   0.10   Jarque-Bera (JB):                 2.31
Prob(Q):                              0.75   Prob(JB):                         0.31
Heteroskedasticity (H):               5.07   Skew:                             0.03
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.30
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Observation: ARIMA orders with p and q from 0 to 3 were compared for d = 2, and ARIMA(1, 2, 2) has the lowest AIC (1054.89). The AR coefficient (-0.5137) is significant (p < 0.001), but the two MA coefficients have very large standard errors (13.7 and 12.5) and p-values near 1, so they are not reliable individually. The residual checks are reasonable: the Ljung-Box test (p = 0.75) shows no leftover autocorrelation and the Jarque-Bera test (p = 0.31) is consistent with normal residuals, but the heteroskedasticity test (p = 0.00) shows that the error variance changes over time.

Task 11: Forecast the test period

In [10]:

```
# ---- Task 11: forecast the test period ----
forecast = model.forecast(steps=len(test)); forecast.index = test.index
print("\nTask 11: Actual vs forecast (first 10 test months):")
print(pd.DataFrame({'Actual': test, 'Forecast': forecast.round(1)}).head(10).to_string())
```

Out [10]:

```
Task 11: Actual vs forecast (first 10 test months):
            Actual  Forecast
Month
1958-08-01     505     505.3
1958-09-01     404     503.1
1958-10-01     359     509.4
1958-11-01     310     511.3
1958-12-01     337     515.5
1959-01-01     360     518.5
1959-02-01     342     522.1
1959-03-01     406     525.4
1959-04-01     396     528.9
1959-05-01     420     532.3
```

Observation: The forecast starts at 505.3 and rises slowly and smoothly (532.3 by May 1959). The actual passengers fall after the August 1958 peak (505 to 310 in November) and then recover, so the forecast misses the seasonal pattern.

Task 12: Compute MSE, MAE and RMSE

In [11]:

```
# ---- Task 12: evaluation metrics ----
mse = mean_squared_error(test, forecast); mae = mean_absolute_error(test, forecast); rmse = np.sqrt(mse)
mape = (np.abs((test - forecast) / test).mean()) * 100
print("\nTask 12: Forecast accuracy on the test set")
print(f"  MSE  : {mse:.2f}"); print(f"  MAE  : {mae:.2f}"); print(f"  RMSE : {rmse:.2f}"); print(f"  MAPE : {mape:.2f}%")
```

Out [11]:

```
Task 12: Forecast accuracy on the test set
  MSE  : 16799.32
  MAE  : 115.32
  RMSE : 129.61
  MAPE : 29.05%
```

Observation: The model has MSE 16,799.32, MAE 115.32, RMSE 129.61 and MAPE 29.05%. On average the forecast is about 115 thousand passengers (29%) away from the actual value, so the accuracy is low for this data.

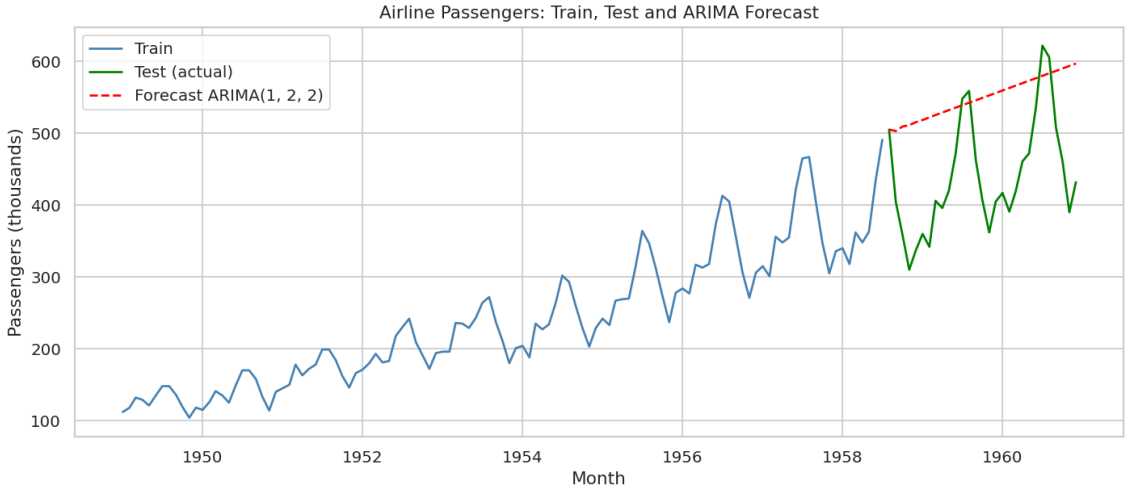
Task 13: Plot train, test and predictions in a single plot

In [12]:

```
# ---- Task 13: train, test and predictions in a single plot ----
plt.figure(figsize=(11, 5))
plt.plot(train, label='Train', color='steelblue')
plt.plot(test, label='Test (actual)', color='green')
plt.plot(forecast, label=f'Forecast ARIMA{order}', color='red', linestyle='--')
plt.title('Airline Passengers: Train, Test and ARIMA Forecast'); plt.xlabel('Month'); plt.ylabel('Passengers (thousands)')
plt.legend(); plt.tight_layout(); plt.show()
```

Out [12]:

(forecast plot shown below)



> **Observation:** The forecast is a nearly straight rising line that follows the trend only. It does not reproduce the summer peaks and winter dips of the test period. The reason is that ARIMA has no seasonal component, while this series has strong yearly seasonality. A seasonal model such as SARIMA with a period of 12 months (or ARIMA on the log of the series) would be expected to do much better.

Result

Thus the monthly airline passenger series (144 months) was analysed. The ADF test showed that the original series is non-stationary (p = 0.9919), the first difference is still non-stationary at the 5% level (p = 0.0542), and the second difference is stationary (p about 0), so d = 2 was used. An ARIMA(1, 2, 2) model, chosen by lowest AIC, was fitted on the first 115 months and used to forecast the last 29 months. The forecast errors were MSE 16,799.32, MAE 115.32 and RMSE 129.61. The forecast follows the trend but misses the yearly seasonality, so a seasonal model such as SARIMA is recommended to improve accuracy.